4.2

Каналы, select и context глубже

Как компактный Go-код превращается в копирование, локи, очереди ожидания и распространение отмены.

```
makechan / chansend / chanrecv
selectgo
propagateCancel / cancelCtx / timerCtx
```

Что уже известно и чего здесь не повторяем

ОПОРА

Из Runtime deep уже известны G/M/P, sudog, gopark/goready, парковка и планировщик.

НОВЫЙ СЛОЙ

Отправка и получение: решение до парковки и состояние после пробуждения.
select: регистрация и выбор победителя.
context: физическая передача отмены.

ИНЖЕНЕРНЫЙ ВЫВОД

Смотрим на алгоритмы конструкций, не перечитывая устройство планировщика.

Три расследования одной лекции

МАРШРУТ

```
канал → makechan → chansend / chanrecv / closechan
select → pollOrder → lockOrder → три прохода
context → propagateCancel → cancelCtx → timerCtx
```

ИНЖЕНЕРНЫЙ ВЫВОД

Каждый механизм появляется там, где простая модель перестает объяснять происходящее.

От make(chan T, n) до hchan

Сначала выясним, что рантайм создает еще до первой отправки.

```
тип T → размер / указатели
hchan + кольцевой буфер
физическая стоимость
```

Что выделяет makechan

GO / РАНТАЙМ

```
ch := make(chan Job, 128)
```

ЧТО ПРОИСХОДИТ

Нужны метаданные `hchan` и место под 128 значений `Job`.
Объем = `capacity × sizeof(T)`.
Тип определяет не только размер, но и карту указателей для GC.

ИНЖЕНЕРНЫЙ ВЫВОД

Создание канала — реальная аллокация, зависящая от типа и емкости.

hchan: минимальное состояние

ПОЛЯ

```
qcount   — число элементов
dataqsiz — емкость буфера
buf      — адрес буфера
sendx / recvx — индексы кольца
recvq / sendq — очереди ожидания
closed / lock — состояние и защита
```

ИНЖЕНЕРНЫЙ ВЫВОД

Поля объясняют и быстрые пути, и блокировку. Это детали текущего рантайма.

Два инварианта очередей

ИНВАРИАНТ

`qcount > 0` очередь получателей `recvq` пуста.
`qcount < dataqsiz` очередь отправителей `sendq` пуста.

ПОЧЕМУ

Если значение уже лежит в буфере, получателю незачем ждать.
Если есть место, отправителю незачем ждать.

ИНЖЕНЕРНЫЙ ВЫВОД

Очереди ожидания согласованы с заполненностью буфера.

Безбуферизованный канал

GO / РАНТАЙМ

```
ch := make(chan int)
_ = make(chan struct{})
```

ЧТО ПРОИСХОДИТ

Емкость равна нулю — массива значений нет.
Для типа нулевого размера отдельное хранилище элементов тоже не нужно.

ИНЖЕНЕРНЫЙ ВЫВОД

При нулевом размере данных рантайм не обязан выделять отдельный буфер.

Тип элемента меняет схему аллокации

БЕЗ УКАЗАТЕЛЕЙ

make(chan [64]byte, 128)
буфер без Go-указателей
hchan и буфер могут жить в одной аллокации

С УКАЗАТЕЛЯМИ

make(chan *Node, 128)
буфер содержит указатели
отдельная типизированная память для GC

ИНЖЕНЕРНЫЙ ВЫВОД

Разная организация памяти при одинаковом API make(chan T, n).

Большой буфер — большая память

GO / РАНТАЙМ

```
ch := make(chan [4096]byte,
10_000)
```

ЧТО ПРОИСХОДИТ

Под значения потребуется около 39 МиБ.
Если элементы содержат указатели, GC также должен учитывать их при сканировании.

ИНЖЕНЕРНЫЙ ВЫВОД

Большая емкость резервирует память заранее — даже если отправок пока нет.

chansend: где выбирается путь

От одной строки ch <- v до прямой передачи, записи в буфер или парковки.

```
nil → быстрый отказ
лок → получатель? → буфер?
ожидание → возобновление
```

Что получает chansend

GO / РАНТАЙМ

```
ch <- value
```

ЧТО ПРОИСХОДИТ

`c` — указатель на `hchan`.
`ep` — адрес значения.
`block` — допускается ли ожидание.
Компилятор уже подготовил эти данные.

ИНЖЕНЕРНЫЙ ВЫВОД

Рантайм получает не синтаксис Go, а структуру канала и адрес элемента.

nil-канал: hchan не существует

ЗАДАЧА

Отправка в `nil` при блокирующем вызове не может завершиться.

МЕХАНИЗМ

Блокирующий путь парковка без последующего пробуждения. Неблокирующий вернуть `false`.

ИНЖЕНЕРНЫЙ ВЫВОД

Ни лока, ни очереди ожидания нет: объекта канала тоже нет.

Быстрый отказ без лока

ЗАДАЧА

Для неблокирующей отправки канал открыт, но отправить сейчас невозможно.

МЕХАНИЗМ

`closed == 0` и `full(c)` дают основание сразу вернуть `false`, не захватывая `hchan.lock`.

ИНЖЕНЕРНЫЙ ВЫВОД

Иногда достаточно допустимого наблюдения состояния, а не атомарного снимка всей структуры.

Почему быстрый путь не требует идеального снимка

ЗАДАЧА

Состояние меняется конкурентно; два поля читаются не как единая транзакция.

МЕХАНИЗМ

Корректность строится на разрешенном моменте наблюдения и свойствах переходов состояния канала.

ИНЖЕНЕРНЫЙ ВЫВОД

Оптимизация корректна по протоколу, даже если нельзя остановить весь мир для чтения.

Основной порядок chansend

ДЕРЕВО РЕШЕНИЙ

```
nil?
неблокирующий отказ?
lock(hchan) → closed?
recvq: есть получатель? → прямая передача
иначе буфер свободен? → запись по sendx
иначе sendq + sudog + парковка
```

ИНЖЕНЕРНЫЙ ВЫВОД

Ожидающий получатель важнее записи в свободный буфер.

Отправка в закрытый канал

GO / РАНТАЙМ

```
ch <- value
```

ЧТО ПРОИСХОДИТ

Под локом рантайм проверяет `closed`.
Если канал закрыт, операция завершается `panic("send on closed channel")`.

ИНЖЕНЕРНЫЙ ВЫВОД

Закрытие проверяется до выбора обычной отправки или регистрации ожидания.

Почему прямой получатель важнее буфера

ЛИШНЯЯ РАБОТА

Можно: отправитель буфер
получатель.
Две передачи через промежуточную ячейку.

КОРОТКИЙ ПУТЬ

Получатель уже в `recvq`.
Можно сразу: отправитель
получатель.
Буфер пропускаем.

ИНЖЕНЕРНЫЙ ВЫВОД

При ожидающем получателе текущий chansend использует прямую передачу.

Небуферизованный канал связывает два стека

GO / РАНТАЙМ

```
ch := make(chan Job)

// G2 ждет: job := <-ch
// G1 делает: ch <- value
```

ЧТО ПРОИСХОДИТ

Значение может находиться на стеке G1.
Место назначения — на стеке ожидающей G2.
Рантайм передает байты напрямую.

ИНЖЕНЕРНЫЙ ВЫВОД

Отправка не обязана создавать объект в хипе и не обязана использовать буфер канала.

sendDirect: прямое копирование

ПЕРЕДАЧА

```
стек отправителя G1
↓ адрес ep
sendDirect
↓ typeBitsBulkBarrier + memmove
стек получателя G2
↓ sg.success = true → goready(G2)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Рантайм в особом протоколе может записывать в стек другой горутины.

Почему запись в чужой стек особенная

ЗАДАЧА

Стек ожидающей G может перемещаться.
`sg.elem` хранит адрес внутри такого стека.

МЕХАНИЗМ

Рантайм ограничивает перемещение стека на критическом участке и применяет специальные операции копирования с учетом GC.

ИНЖЕНЕРНЫЙ ВЫВОД

Обычный memmove без протокола работы со стеком и барьером не является общей реализацией.

Тип T с указателями: сохраняем инварианты GC

ЗАДАЧА

В значении могут быть Go-указатели; GC маркирует одновременно с работой программы.

МЕХАНИЗМ

Прямая передача использует `typeBitsBulkBarrier`, затем `memmove` между стеками.

ИНЖЕНЕРНЫЙ ВЫВОД

Перенос байтов и корректность графа указателей — две отдельные задачи.

Буферизованная отправка

КОЛЬЦЕВОЙ БУФЕР

```
ожидающий получатель отсутствует
qcount < dataqsiz
qp = chanbuf(c, sendx)
typedmemmove(elemtype, qp, ep)
sendx = (sendx + 1) по кольцу
qcount++ → unlock
```

ИНЖЕНЕРНЫЙ ВЫВОД

Работа относительно длины буфера O(1); вся очередь не сдвигается.

Индексы sendx / recvx

ЗАДАЧА

`sendx` выбирает следующую ячейку записи.
`recvx` — следующую ячейку чтения.

МЕХАНИЗМ

При емкости 4: `0 → 1 → 2 → 3 → 0`.
Адрес вычисляется как `buf + index × elemsize`.

ИНЖЕНЕРНЫЙ ВЫВОД

Индекс перемещается; сами элементы массива при обычном чтении не сдвигаются.

Почему qcount хранится отдельно

ЗАДАЧА

При `sendx == recvx` кольцо может быть и пустым, и полностью заполненным.

МЕХАНИЗМ

`qcount` различает состояния. `0` пусто; `dataqsiz` заполнено.

ИНЖЕНЕРНЫЙ ВЫВОД

По одной разности индексов число элементов надежно не вывести.

Заблокированная отправка

МЕДЛЕННЫЙ ПУТЬ

```
получателя нет
буфер заполнен
acquireSudog → sg.elem = адрес значения
sendq.enqueue(sg)
gopark → G ожидает
```

ИНЖЕНЕРНЫЙ ВЫВОД

Значение не копируется в специальный объект в хипе для ожидания: его адрес сохраняется в sudog.

Зачем sg.elem указывает на значение

ЗАДАЧА

Отправитель припаркован, а получатель придет позже.

МЕХАНИЗМ

Получатель сможет взять данные прямо из известного слота отправителя.

ИНЖЕНЕРНЫЙ ВЫВОД

Экономия копирования требует согласования времени жизни адреса со стеком G.

После пробуждения отправитель проверяет результат

ЗАДАЧА

G могла проснуться после передачи значения либо из-за закрытия канала.

МЕХАНИЗМ

`sudog.success == true` отправка завершена.
`false` после close `panic` в возобновившемся `chansend`.

ИНЖЕНЕРНЫЙ ВЫВОД

Точка возникновения panic остается внутри операции отправителя.

chanrecv: симметрия с особенностями

Буфер с ожидающим отправителем обнаруживает более сложный алгоритм, чем простое чтение.

```
nil / empty / closed
ожидающий отправитель
заполненный буфер: обмен
```

Что получает chanrecv

GO / РАНТАЙМ

```
v, ok := <-ch
```

ЧТО ПРОИСХОДИТ

`c` — канал; `ep` — место для элемента; `block` — можно ли ожидать.
Результат включает факт получения и готовность операции.

ИНЖЕНЕРНЫЙ ВЫВОД

Уже в рантайме есть различие «операция завершилась» и «получено значение».

Неблокирующее получение и порядок наблюдений

ЗАДАЧА

Буфер может перейти через состояния: открыт+данные закрыт+данные закрыт+пуст.

МЕХАНИЗМ

Проверка `empty` и `closed` требует корректного порядка атомарных наблюдений.

ИНЖЕНЕРНЫЙ ВЫВОД

Неправильно составленный снимок способен нарушить семантику чтения после close.

Закрыт и пуст: нулевое значение + ok=false

GO / РАНТАЙМ

```
v, ok := <-ch
```

ЧТО ПРОИСХОДИТ

`closed == 1` и `qcount == 0` обнулить место назначения. Результат `ok == false`.

ИНЖЕНЕРНЫЙ ВЫВОД

Закрытие не заставляет возвращать false до опустошения буфера.

Закрытый канал может хранить данные

НЕВЕРНО

Неверно: close(ch) каждое чтение сразу возвращает нулевое значение.

КАК ЕСТЬ

После `close` буфер `[A][B][C]` сначала выдаст A, B, C.
И только потом — zero, false.

ИНЖЕНЕРНЫЙ ВЫВОД

closed и empty — независимые признаки состояния канала.

Ожидающий отправитель: два варианта

RECV

```
sendq.dequeue()
↓
dataqsiz == 0?
├─ да → recvDirect со стека отправителя
└─ нет → буфер заполнен
          старый head → receiver
          sender value → тот же slot
```

ИНЖЕНЕРНЫЙ ВЫВОД

Один путь обмена нужен для небуферизованного канала, другой — для заполненного буфера.

recvDirect: зеркальный протокол

ЗАДАЧА

Получатель работает сейчас, отправитель ожидает в `sendq`.

МЕХАНИЗМ

`sg.elem` указывает на значение в стеке отправителя. `recvDirect` делает барьер GC и копирует данные.

ИНЖЕНЕРНЫЙ ВЫВОД

Ожидающий стек защищен специальным протоколом рантайма на время передачи.

Самый интересный случай: полный буфер

СОСТОЯНИЕ ДО / ПОСЛЕ

```
capacity = 4, qcount = 4
буфер [A][B][C][D]
sendq: G5 хочет отправить E
пришел получатель
→ A отдаем receiver
→ E кладем на место A
```

ИНЖЕНЕРНЫЙ ВЫВОД

Одно чтение одновременно обслуживает ожидающего отправителя.

Полный буфер: зачем обмен в одной секции

ДВА ЭТАПА

Наивно: взять A разбудить G5 он позже получит лок запишет E.

ОДИН ЭТАП

В текущем recv: под одним локом A идет получателю, E занимает освобожденную ячейку.

ИНЖЕНЕРНЫЙ ВЫВОД

Меньше повторных захватов лока и ненужных переходов через планировщик.

Почему sendx == recvx при полном кольце

ЗАДАЧА

Головная ячейка одновременно совпадает с позицией новой записи после удаления первого элемента.

МЕХАНИЗМ

При обмене в заполненном кольце старая ячейка A заменяется значением E, `recvx` сдвигается, `sendx` следует за ним.

ИНЖЕНЕРНЫЙ ВЫВОД

qcount остается равным dataqsiz: один элемент ушел и один сразу пришел.

Обычное получение из буфера

ПУТЬ ИЗ БУФЕРА

```
ожидающего отправителя нет
qcount > 0
qp = chanbuf(c, recvx)
typedmemmove(elemtype, ep, qp)
typedmemclr(elemtype, qp)
recvx по кольцу → qcount--
```

ИНЖЕНЕРНЫЙ ВЫВОД

Освобожденную типизированную ячейку очищаем: старые указатели не должны удерживать объекты.

Зачем очищать ячейку после чтения

ЗАДАЧА

Значение в ячейке логически уже удалено, но ее bytes могут содержать `*User`.

МЕХАНИЗМ

`typedmemclr` обнуляет ячейку с учетом типа.

ИНЖЕНЕРНЫЙ ВЫВОД

Не оставляем ложных ссылок, которые GC мог бы продолжать считать достижимыми.

Когда chanrecv паркует G

МЕДЛЕННЫЙ ПУТЬ

```
nil → блокировка навсегда
канал пуст, отправителей нет
↓
acquireSudog
sg.elem = место назначения получателя
recvq.enqueue(sg) → gopark
```

ИНЖЕНЕРНЫЙ ВЫВОД

При получении очередь используется только тогда, когда нельзя забрать ни значение из буфера, ни ожидающего отправителя.

Полное дерево получения

ДЕРЕВО РЕШЕНИЙ

```
nil? → особый случай
неблокирующий быстрый путь
lock → закрыт и пуст? → zero, false
ожидающий отправитель? → direct / full-buffer обмен
в буфере есть данные? → обычное чтение
иначе recvq + sudog + парковка
```

ИНЖЕНЕРНЫЙ ВЫВОД

Результат определяется не одним флагом, а сочетанием закрытия, очереди отправителей и заполненности.

closechan: одна операция — много ожиданий

Закрытие должно согласованно освободить две очереди с разной семантикой.

```
closed = 1
recvq → zero / false
sendq → будущая panic
```

Что проверяет closechan

GO / РАНТАЙМ

```
close(ch)
```

ЧТО ПРОИСХОДИТ

`nil` panic.
Повторное закрытие panic.
Под `hchan.lock` устанавливается `closed = 1`.

ИНЖЕНЕРНЫЙ ВЫВОД

После изменения состояния необходимо разобрать все зарегистрированные ожидания.

Ожидающие получатели

ЗАДАЧА

В `recvq` могут находиться G, которые ждут данные.

МЕХАНИЗМ

Для них `closechan` очищает место назначения, устанавливает `sg.success = false`, готовит пробуждение.

ИНЖЕНЕРНЫЙ ВЫВОД

Возобновившийся receive вернет нулевое значение и `ok=false`.

Ожидающие отправители

ЗАДАЧА

Отправитель пытается завершить `ch <- value`, хотя канал закрыт.

МЕХАНИЗМ

`closechan` помечает ожидание как неуспешное и делает G готовой.

ИНЖЕНЕРНЫЙ ВЫВОД

Panic выполняется позже внутри возобновившегося chansend, а не на чужом стеке.

Почему пробуждение происходит после unlock

ПОРЯДОК

```
lock(hchan)
closed = 1
разобрать recvq и sendq
собрать G в glist
unlock(hchan)
goready для всех G
```

ИНЖЕНЕРНЫЙ ВЫВОД

Под локом меняем защищаемое состояние; действия по пробуждению делаем после его освобождения.

Почему нельзя будить G прямо под hchan.lock

ЗАДАЧА

Управление состоянием другой G может пересечься со stack shrinking и планированием.

МЕХАНИЗМ

Текущий closechan сначала собирает список горутин, потом отпускает лок.

ИНЖЕНЕРНЫЙ ВЫВОД

Более короткая критическая секция и меньше рисков взаимной блокировки.

Стоимость close зависит от очередей

ЗАДАЧА

Пусть в `recvq` ждут 50 000 горутин.

МЕХАНИЗМ

`closechan` обходит ожидающих и подготавливает их; после unlock следует массовое пробуждение.

ИНЖЕНЕРНЫЙ ВЫВОД

Логически один close, но реальная работа O(N) по числу ожидающих.

Модель каналов после close

КОНТРАКТ

```
отправка в закрытый → panic
чтение из закрытого с данными → остаток буфера
чтение из закрытого пустого → zero, false
ждущий sender → проснется и выполнит panic
ждущий receiver → проснется и получит zero, false
```

ИНЖЕНЕРНЫЙ ВЫВОД

Семантика языка сохраняется за счет разных результатов для двух очередей.

selectgo: один победитель из многих

Одна G может ожидать несколько каналов, но завершить ровно одну операцию.

```
pollOrder + lockOrder
первый проход → готовность
второй → регистрация; третий → очистка
```

Общий select работает с несколькими каналами

GO / РАНТАЙМ

```
select {
case v := <-a:
    use(v)
case b <- x:
    sent()
case <-c:
    done()
}
```

ЧТО ПРОИСХОДИТ

Нужно проверить готовность, взять локи без дедлока, зарегистрироваться, выбрать победителя и удалить проигравших.

ИНЖЕНЕРНЫЙ ВЫВОД

Select — алгоритм с несколькими очередями, а не случайный if.

Компилятор готовит данные заранее

ЗАДАЧА

Рантайм получает описания операций и адреса значений; временные массивы порядков создаются на стеке G.

МЕХАНИЗМ

Вход `selectgo`: операции по каналам и память под `pollOrder` и `lockOrder`.

ИНЖЕНЕРНЫЙ ВЫВОД

Как компилятор формирует вызов — отдельная тема 4.3.

Простой select не обязан вызвать selectgo

ПО СИНТАКСИСУ

`select` с одним case уже похож на общий механизм выбора.

ПОСЛЕ КОМПИЛЯТОРА

Компилятор может превратить один case в обычную отправку/получение; case+default — в неблокирующий путь.

ИНЖЕНЕРНЫЙ ВЫВОД

Общий selectgo нужен не для каждой синтаксической конструкции select.

Почему у select два порядка

ДВА РАЗНЫХ ВОПРОСА

```
pollOrder → в каком порядке проверять готовность
lockOrder → в каком порядке брать локи каналов
оба строятся для одного набора case
```

ИНЖЕНЕРНЫЙ ВЫВОД

Случайность нужна выбору; стабильный порядок — защите от взаимной блокировки.

pollOrder: ни одна ветка не должна быть первой всегда

ЗАДАЧА

При текстовом обходе первый готовый case получает систематическое преимущество.

МЕХАНИЗМ

Рантайм строит псевдослучайную перестановку активных веток.

ИНЖЕНЕРНЫЙ ВЫВОД

Это устраняет постоянный перекос, но не обещает строгую очередность.

Как строится pollOrder

ПЕРЕСТАНОВКА

```
уже добавлены [A B C]
добавить D
случайный индекс j из 0..3
при j=1 → [A D C B]
```

ИНЖЕНЕРНЫЙ ВЫВОД

Перемешивание происходит инкрементально, без отдельного большого массива.

Nil-канал исключается из выбора

ЗАДАЧА

Ветка с nil-каналом никогда не может стать готовой.

МЕХАНИЗМ

Рантайм убирает такие ветки из рабочих `pollOrder`/`lockOrder` и очищает ненужную ссылку на значение.

ИНЖЕНЕРНЫЙ ВЫВОД

Нет смысла брать лок несуществующего канала и регистрировать ожидание.

Почему pollOrder нельзя использовать для локов

ПОТЕНЦИАЛЬНЫЙ ДЕДЛОК

G1 берет AB.
G2 берет BA.
Они могут ждать друг друга вечно.

ГЛОБАЛЬНЫЙ ПОРЯДОК

Для любого набора каналов все select берут локи в согласованном `lockOrder`.

ИНЖЕНЕРНЫЙ ВЫВОД

Случайный порядок проверки и безопасный порядок захвата решают разные задачи.

lockOrder опирается на ключ канала

ЗАДАЧА

Один и тот же hchan может участвовать в нескольких конкурирующих select.

МЕХАНИЗМ

Рантайм сортирует записи по ключу hchan, затем захватывает локи в одном порядке.

ИНЖЕНЕРНЫЙ ВЫВОД

Убираем цикл ожидания локов между конкурентными select.

Почему в lockOrder пирамидальная сортировка

ЗАДАЧА

Нужно упорядочить N веток, не создавая много временной памяти.

МЕХАНИЗМ

Текущая реализация использует пирамидальную сортировку: `O(N log N)` и постоянный дополнительный объем стека.

ИНЖЕНЕРНЫЙ ВЫВОД

Алгоритм сортировки — деталь реализации, а отсутствие лишних аллокаций — практическая цель.

Один и тот же канал в двух ветках

GO / РАНТАЙМ

```
select {
case <-ch:
case ch <- x:
}
```

ЧТО ПРОИСХОДИТ

`lockOrder` содержит обе ветки, но `sellock` берет один и тот же лок канала только один раз.

ИНЖЕНЕРНЫЙ ВЫВОД

Нерекурсивный лок нельзя захватывать повторно самой же G.

Три прохода selectgo

ОБЩИЙ АЛГОРИТМ

```
захватить локи по lockOrder
 найти уже готовый case
 иначе зарегистрировать sudog и припарковать G
событие → selectDone выбирает победителя
 очистить остальные ожидания
```

ИНЖЕНЕРНЫЙ ВЫВОД

Главная схема selectgo: проверить зарегистрировать очистить.

Проход 1: проверить готовность

ЗАДАЧА

Локи участвующих каналов уже удерживаются.

МЕХАНИЗМ

По `pollOrder` рантайм проверяет отправителей/получателей, буфер и закрытие.

ИНЖЕНЕРНЫЙ ВЫВОД

Если операция готова, выполняем сразу, без создания sudog.

Почему нельзя регистрировать ожидание до проверки

ЛИШНЯЯ ПАРКОВКА

Данные уже готовы, но G все равно ставит sudog и паркуется.

ПРОХОД 1

Сначала ищем готовую ветку. Только если все заблокированы — переходим к регистрации.

ИНЖЕНЕРНЫЙ ВЫВОД

Быстрый путь избегает работы планировщика и очередей ожидания.

Проход 1: получение

ПРОВЕРКИ

```
ожидающий отправитель?
↓ иначе
буфер содержит значение?
↓ иначе
канал закрыт?
↓ иначе
эта ветка сейчас не готова
```

ИНЖЕНЕРНЫЙ ВЫВОД

При полном буфере и ждущем отправителе используется уже известный обмен головной ячейки с ожидающим отправителем.

Проход 1: отправка

ПРОВЕРКИ

```
канал закрыт? → panic
иначе ждущий получатель? → прямая передача
иначе есть место в буфере? → запись
иначе case не готов
```

ИНЖЕНЕРНЫЙ ВЫВОД

Для отправки закрытие проверяется до обычных путей передачи.

default обрывает алгоритм до парковки

GO / РАНТАЙМ

```
select {
case <-ch:
default:
    fallback()
}
```

ЧТО ПРОИСХОДИТ

Если ничего не готово, рантайм отпускает локи и выбирает `default`.
Регистраций sudog не будет.

ИНЖЕНЕРНЫЙ ВЫВОД

Неблокирующий select платит за проверку, но не за ожидание.

Проход 2: одна G — несколько sudog

ЗАДАЧА

Select с четырьмя активными ветками может зарегистрировать четыре sudog.

МЕХАНИЗМ

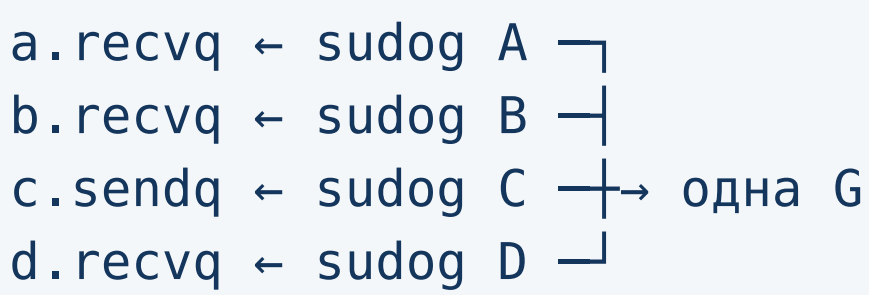
Каждая запись попадает в соответствующую `sendq`/`recvq`, а ссылки на них объединяются в `gp.waiting`.

ИНЖЕНЕРНЫЙ ВЫВОД

Много ожиданий связаны с одной G, которую можно пробудить только один раз.

Регистрация в нескольких очередях

ОДНА G



ИНЖЕНЕРНЫЙ ВЫВОД

Список `gp.waiting` нужен и для безопасной парковки, и для последующей очистки проигравших веток.

Зачем gp.waiting строится по lockOrder

ЗАДАЧА

При регистрации очереди ссылаются на несколько hchan, а список ожиданий принадлежит одной G.

МЕХАНИЗМ

`selparkcommit` позже проходит по `gp.waiting` и отпускает соответствующие локи без повторного освобождения.

ИНЖЕНЕРНЫЙ ВЫВОД

Единый порядок связывает регистрацию, парковку и разблокировку.

Потерянное пробуждение

ОШИБОЧНЫЙ АЛГОРИТМ

Проверили пустой канал сняли лок отправитель пришел никого не увидел ушел мы встали в очередь.

ЗАЩИЩЕННЫЙ АЛГОРИТМ

Проверка условия и регистрация ожидания выполняются под локами каналов как единый протокол.

ИНЖЕНЕРНЫЙ ВЫВОД

Нельзя оставлять окно между проверкой и добавлением waiter.

После регистрации нельзя просто отпустить локи и park

ЗАДАЧА

Другая горутина может успеть обнаружить sudog и вызвать `goready` до завершения парковки G.

МЕХАНИЗМ

Нужен согласованный commit-протокол перехода в WAITING и освобождения нескольких локов.

ИНЖЕНЕРНЫЙ ВЫВОД

Эту узкую границу обеспечивает специальная функция `selparkcommit`.

selparkcommit: переход к ожиданию

СОГЛАСОВАНИЕ

```
gopark(selparkcommit, ...)
публикация stack/channel state
фиксируем корректный переход G в ожидание
отпускаем локи каналов
другая G теперь может обнаружить sudog
```

ИНЖЕНЕРНЫЙ ВЫВОД

Парковка, публикация ссылок на стек и unlock должны образовывать корректный протокол.

activeStackChans: ссылки каналов внутрь стека

ЗАДАЧА

`sudog.elem` может указывать внутрь стека ожидающей G.

МЕХАНИЗМ

`activeStackChans` сообщает механизму стека об активных ссылках из структур каналов.

ИНЖЕНЕРНЫЙ ВЫВОД

Стек нельзя перемещать, игнорируя указатели, которые хранятся вне него.

parkingOnChan: G еще входит в ожидание

ЗАДАЧА

Во время commit рантайм различает фазу постановки G на парковку.

МЕХАНИЗМ

`parkingOnChan` публикует это переходное состояние; после правильной публикации оно сбрасывается.

ИНЖЕНЕРНЫЙ ВЫВОД

Перемещение стека и работа с очередями каналов пересекаются именно на этой границе.

Почему после первого unlock нельзя обращаться к стеку G

ЗАДАЧА

Как только отпущен лок канала, другая G может найти sudog и разбудить ожидающую.

МЕХАНИЗМ

Вскоре эта G уже может исполняться на другом M и пользоваться своим стеком.

ИНЖЕНЕРНЫЙ ВЫВОД

Старый M не должен продолжать трогать стек G после передачи права на выполнение.

Два канала могут стать готовыми одновременно

ЗАДАЧА

G зарегистрирована в `A.recvq` и `B.recvq`.

МЕХАНИЗМ

Два отправителя почти одновременно находят соответствующие записи. Нужен арбитр: победит только одна ветка.

ИНЖЕНЕРНЫЙ ВЫВОД

Просто два раза вызвать goready одной G было бы некорректно.

selectDone: CAS выбирает победителя

ОДИН ПОБЕДИТЕЛЬ

```
waitq.dequeue(sudog)
sg.isSelect?
CAS(g.selectDone, 0 → 1)
├─ успех: эта ветка победила
└─ неуспех: другой case уже выиграл
```

ИНЖЕНЕРНЫЙ ВЫВОД

Арбитраж выполняется при извлечении sudog конкретной операцией канала.

Почему очередь может содержать проигравшую запись

ЗАДАЧА

После победы B sudog A может еще физически находиться в `A.recvq`.

МЕХАНИЗМ

Следующая операция в A обнаружит, что `selectDone` уже установлен, и пропустит устаревший sudog.

ИНЖЕНЕРНЫЙ ВЫВОД

Логическое проигрывание опережает физическую очистку очереди.

gp.param сообщает, какая ветка победила

ЗАДАЧА

Операция канала записывает указатель на победивший sudog в `gp.param`.

МЕХАНИЗМ

После пробуждения `selectgo` использует этот указатель, чтобы определить выбранный case.

ИНЖЕНЕРНЫЙ ВЫВОД

Пробуждение несет результат выбора, а не только сигнал «можно работать».

Проход 3: убрать всех проигравших

ЗАДАЧА

Проснувшаяся G все еще зарегистрирована в других каналах.

МЕХАНИЗМ

Повторно захватываем локи, удаляем проигравшие sudog из `sendq`/`recvq`, очищаем ссылки, освобождаем sudog.

ИНЖЕНЕРНЫЙ ВЫВОД

Без этого оставались бы устаревшие записи и ложные попытки пробуждений.

Очистка после select

ПОСЛЕ ПРОБУЖДЕНИЯ

```
G проснулась → sellock
выделить победивший sudog из gp.param
сбросить ссылки sudog на стек
удалить проигравших из очередей
releaseSudog для всех
unlock → вернуть выбранный case
```

ИНЖЕНЕРНЫЙ ВЫВОД

Проход 3 — часть корректности и стоимости блокирующего select.

Почему очистка обязательна для производительности

ЗАДАЧА

Без удаления проигравших ожиданий очереди копили бы устаревшие sudog.

МЕХАНИЗМ

Будущим отправителям/получателям пришлось бы пропускать все больше «мертвых» записей.

ИНЖЕНЕРНЫЙ ВЫВОД

Цена завершения select включает возвращение структуры к согласованному состоянию.

selectgo целиком

ИТОГОВАЯ СХЕМА

```
pollOrder: порядок проверки
lockOrder: порядок локов
□ готовый case? → сразу выполнить
□ нет default → зарегистрировать N sudog → park
CAS selectDone → один победитель
□ удалить N−1 проигравших
```

ИНЖЕНЕРНЫЙ ВЫВОД

В общей форме select обрабатывает число активных веток, а не работает за константное время.

Почему широкий select дороже

ЗАДАЧА

На N активных веток создаются порядки, проходят проверки и берутся локи.

МЕХАНИЗМ

`pollOrder` ~ O(N); `lockOrder` ~ O(N log N); при парковке N регистраций и O(N) очистка.

ИНЖЕНЕРНЫЙ ВЫВОД

Количество веток — часть стоимости конструкции.

Готовый и блокирующий select — разная цена

ГОТОВ СРАЗУ

Если ветка готова: первый проход нашел ее, выполнил действие, вернулся.

НУЖНА ПАРКОВКА

Если все ждут: второй проход создал ожидания, G припарковалась; третий удалил проигравших.

ИНЖЕНЕРНЫЙ ВЫВОД

Ширина select важна, но характер готовности под нагрузкой — не меньше.

Несколько select на одном горячем канале

ЗАДАЧА

Тысячи G одновременно включают один и тот же `hot` в набор веток.

МЕХАНИЗМ

Все они конкурируют за `hchan.lock` при проверке и регистрации.

ИНЖЕНЕРНЫЙ ВЫВОД

select не устраняет конкуренцию на уровне отдельного канала.

Псевдослучайный выбор — не приоритетный планировщик

ЗАДАЧА

`pollOrder` мешает первой ветке систематически побеждать из-за порядка записи.

МЕХАНИЗМ

Но нет гарантии точных 1/N выборов, SLA-приоритетов или строгой очередности.

ИНЖЕНЕРНЫЙ ВЫВОД

Если нужна бизнес-политика выбора сообщений, задайте ее явно.

context: как отмена движется по дереву

Done — только точка наблюдения; за ней есть связи родителей, дочерних контекстов и таймеры.

```
cancelCtx → propagateCancel
parentCancelCtx / AfterFunc / горутина
cancel → timerCtx
```

Context — не просто канал Done

УПРОЩЕНИЕ

У дочернего контекста есть Done. Значит, наверное, кто-то слушает родителя в отдельной горутине.

ТОЧНАЯ МОДЕЛЬ

Стандартный parent может держать прямую ссылку на child; отдельная горутина используется лишь при определенных реализациях.

ИНЖЕНЕРНЫЙ ВЫВОД

API одинаков, а механизм распространения и его цена зависят от реализации родителя.

cancelCtx: какие поля обеспечивают инварианты

ВНУТРЕННЯЯ СТРУКТУРА

```
Context — родитель
mu — защита связей
done — канал завершения
children — дочерние отменяемые контексты
err / cause — причина отмены
```

ИНЖЕНЕРНЫЙ ВЫВОД

Err, cause, Done и дерево потомков должны меняться согласованно при конкурентных вызовах.

Done хранится через atomic.Value

ЗАДАЧА

Многие контексты ни разу не вызывают `Done()`, а чтение Done на горячем пути должно быть дешевым.

МЕХАНИЗМ

Канал создается лениво; опубликованное значение быстро читается атомарной загрузкой.

ИНЖЕНЕРНЫЙ ВЫВОД

Не нужно выделять канал там, где прикладной код никогда его не наблюдал.

Первый вызов Done()

ЛЕНИВАЯ ИНИЦИАЛИЗАЦИЯ

```
atomic load(done)
канал создан? → сразу вернуть
иначе lock(mu)
проверить еще раз
make(chan struct{}) → atomic store
unlock → вернуть канал
```

ИНЖЕНЕРНЫЙ ВЫВОД

Двойная проверка защищает от двух одновременных аллокаций одного Done.

Отмена до первого Done()

ЗАДАЧА

Контекст отменен, но собственный канал `Done` никогда не создавался.

МЕХАНИЗМ

Рантайм может записать ссылку на общий уже закрытый `closedchan` вместо отдельной аллокации.

ИНЖЕНЕРНЫЙ ВЫВОД

Нулевая полезность уникального канала — нулевая необходимость его создавать.

Err(): быстрый путь чтения

ЗАДАЧА

`Err()` часто проверяется конкурентно и должен быстро показывать результат отмены.

МЕХАНИЗМ

Текущая реализация хранит ошибку в `atomic.Value`; частое чтение не требует общего мьютекса.

ИНЖЕНЕРНЫЙ ВЫВОД

Стоимость регулярной проверки отмены не должна равняться стоимости модификации дерева.

С чего начинается WithCancel

GO / РАНТАЙМ

```
child, cancel :=
context.WithCancel(parent)
```

ЧТО ПРОИСХОДИТ

Создается `cancelCtx` и вызывается `propagateCancel(parent, child)`.
Нужно гарантировать отмену потомка при будущем закрытии родителя.

ИНЖЕНЕРНЫЙ ВЫВОД

Вся сложность — корректно подключить неизвестную реализацию интерфейса Context.

Родителем может быть не только cancelCtx

ЗАДАЧА

`parent` — интерфейс Context, а не обязательно стандартный тип.

МЕХАНИЗМ

Возможны `Background`, `timerCtx`, пользовательская обертка, собственный Context с измененным Done.

ИНЖЕНЕРНЫЙ ВЫВОД

propagateCancel должен соблюдать контракт произвольного Context.

Первая ветка: parent.Done() == nil

САМЫЙ ДЕШЕВЫЙ ПУТЬ

```
вызвать parent.Done()
результат nil?
↓ да
родитель не передает сигнал отмены
ничего не регистрировать → вернуть
```

ИНЖЕНЕРНЫЙ ВЫВОД

Для Background/WithoutCancel не нужны ни children-map, ни наблюдатель, ни обратный вызов.

Вторая ветка: родитель уже отменен

РАННЯЯ ОТМЕНА

```
parent.Done() != nil
неблокирующая проверка <-parent.Done()
канал уже закрыт?
→ child.cancel(...)
→ вернуть
```

ИНЖЕНЕРНЫЙ ВЫВОД

> Не регистрируем потомка у родителя, когда событие уже произошло.

Стандартный parent: быстрый путь

ЗАДАЧА

Потомок создан от обычного `context.WithCancel`/`WithTimeout`.

МЕХАНИЗМ

`parentCancelCtx` находит внутренний `*cancelCtx`; под `parent.mu` регистрируется `children[child]`.

ИНЖЕНЕРНЫЙ ВЫВОД

Никакая горутина-наблюдатель для связи parentchild не нужна.

parentCancelCtx: регистрация ребенка

ОБЫЧНЫЙ ПУТЬ

```
найти внутренний *cancelCtx
lock(parent.mu)
родитель уже отменен? → отменить child
иначе создать children map при необходимости
children[child] = struct{}{}
unlock(parent.mu)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Согласование состояния отмены и регистрации происходит под одним локом родителя.

Как найти cancelCtx через интерфейс

ЗАДАЧА

Context не имеет публичного метода «верни внутренний cancelCtx».

МЕХАНИЗМ

Стандартная библиотека использует особый ключ `Value(&cancelCtxKey)`.

ИНЖЕНЕРНЫЙ ВЫВОД

Внутренний протокол позволяет пройти через прозрачные обертки, не меняя публичный API.

Почему недостаточно только Value(&cancelCtxKey)

РИСК ОБХОДА

Обертка может делегировать Value, но вернуть собственный Done-канал.

ЗАЩИТА

Библиотека дополнительно проверяет, что Done родителя совпадает с Done найденного cancelCtx.

ИНЖЕНЕРНЫЙ ВЫВОД

Иначе можно обойти переопределенную семантику пользовательской обертки.

children map тоже создается лениво

ЗАДАЧА

У cancelCtx может не быть ни одного потомка.

МЕХАНИЗМ

Map появляется при первой регистрации child; до этого дополнительное хранилище не нужно.

ИНЖЕНЕРНЫЙ ВЫВОД

Миллионы контекстов без детей не оплачивают пустую структуру потомков.

Широкое дерево потомков имеет реальную стоимость

ЗАДАЧА

У одного родителя 100 000 зарегистрированных потомков.

МЕХАНИЗМ

Map удерживает ссылки. При отмене ее надо обойти и отменить каждый child.

ИНЖЕНЕРНЫЙ ВЫВОД

Стандартный путь без наблюдателей дешевле, но не O(1) при большом числе потомков.

Следующий путь: parent поддерживает AfterFunc

ВТОРАЯ СТРАТЕГИЯ

```
прямой cancelCtx не найден
parent реализует AfterFunc?
зарегистрировать функцию обратного вызова
при отмене parent: callback → child.cancel
сохранить stop-функцию в stopCtx
```

ИНЖЕНЕРНЫЙ ВЫВОД

Регистрация обратного вызова позволяет не создавать постоянно работающую горутину-наблюдатель.

Зачем нужен stopCtx

ЗАДАЧА

Дочерний контекст мог завершиться раньше родителя.

МЕХАНИЗМ

При `removeChild` нужно остановить обратный вызов, зарегистрированный в AfterFunc родителя.

ИНЖЕНЕРНЫЙ ВЫВОД

Раннее завершение потомка должно разрывать неиспользуемые связи.

Самый дорогой резервный путь

РЕЗЕРВНЫЙ ПУТЬ

```
parent.Done() != nil
нет распознаваемого cancelCtx
нет AfterFunc
↓
создать горутину-наблюдатель
select { parent.Done(); child.Done() }
```

ИНЖЕНЕРНЫЙ ВЫВОД

Универсальный контракт Context сохраняется ценой отдельной горутины.

Горутина-наблюдатель должна завершаться по двум событиям

GO / РАНТАЙМ

```
go func() {
    select {
    case <-parent.Done():
        cancelChild()
    case <-child.Done():
        return
    }
}()
```

ЧТО ПРОИСХОДИТ

Родитель отменился отменить child.
Child отменился самостоятельно остановить наблюдателя.

ИНЖЕНЕРНЫЙ ВЫВОД

Без второго case наблюдатель мог бы остаться жить до отмены родителя.

Один API — три физические цены

БЕЗ ГОРУТИНЫ

Стандартный cancelCtx запись в children map.
Parent с AfterFunc регистрация функции обратного вызова.

РЕЗЕРВНЫЙ ПУТЬ

Пользовательский Context без обоих механизмов отдельная горутина на child.

ИНЖЕНЕРНЫЙ ВЫВОД

Цена WithCancel зависит от механизма конкретного родительского Context.

Почему собственная обертка Context может удорожить код

ЗАДАЧА

Обертка может изменить `Done()` и тем самым отключить распознавание `cancelCtx`.

МЕХАНИЗМ

propagateCancel перейдет к AfterFunc или к наблюдателю.

ИНЖЕНЕРНЫЙ ВЫВОД

Абстракция с тем же API способна существенно изменить профиль числа горутин.

propagateCancel: дерево решений целиком

ЧЕТЫРЕ ВЕТКИ

```
parent.Done() == nil? → ничего не делать
родитель уже отменен? → cancel child
parentCancelCtx? → children[child]
AfterFunc? → callback + stopCtx
иначе → горутина-наблюдатель
```

ИНЖЕНЕРНЫЙ ВЫВОД

От самого дешевого случая к универсальному резервному: стоимость растет по мере потери сведений о родителе.

cancelCtx.cancel: что делает cancel()

Отмена — это изменение состояния, закрытие Done и синхронный обход дерева.

```
mu → err/cause
close(Done) → children.cancel
removeChild / timer.Stop
```

Первый cancel устанавливает состояние

ПОРЯДОК

```
lock(cancelCtx.mu)
err уже установлен? → вернуть
иначе сохранить err / cause
закрыть Done или использовать closedchan
отменить children → children = nil
unlock
```

ИНЖЕНЕРНЫЙ ВЫВОД

Повторные вызовы безопасны: реальную работу выполняет первый.

Повторная отмена не повторяет работу

GO / РАНТАЙМ

```
cancel()
cancel()
cancel()
```

ЧТО ПРОИСХОДИТ

При повторном вызове `err != nil`: контекст уже отменен, закрывать канал и обходить дерево снова нельзя.

ИНЖЕНЕРНЫЙ ВЫВОД

Идемпотентность cancel гарантирует отсутствие panic от повторного close(Done).

Cause: причина появляется вместе с Err

ЗАДАЧА

Обычный CancelFunc и CancelCauseFunc должны согласованно зафиксировать причину.

МЕХАНИЗМ

Первый cancel устанавливает `err` и `cause`; если отдельная причина не передана, `cause = err`.

ИНЖЕНЕРНЫЙ ВЫВОД

Потомки получают согласованные сведения об отмене.

Отмена потомков

ОБХОД ДЕРЕВА

```
родитель хранит children map
при cancel под parent.mu:
для каждого child → child.cancel(false, err, cause)
дочерний контекст рекурсивно отменяет своих детей
после обхода children = nil
```

ИНЖЕНЕРНЫЙ ВЫВОД

Стоимость зависит от размера отменяемого поддерева.

Почему removeChild вызывается после unlock

ИНВЕРСИЯ ЛОКОВ

Опасно: держать child.mu и ждать parent.mu.
Распространение идет parent.mu child.mu.

БЕЗОПАСНЫЙ ПОРЯДОК

Сначала отпустить child.mu, затем удалить запись у parent.

ИНЖЕНЕРНЫЙ ВЫВОД

Противоположный порядок захвата локов мог бы создать взаимную блокировку.

Почему children = nil после отмены

ЗАДАЧА

Родитель уже отменен, второй раз распространять сигнал не нужно.

МЕХАНИЗМ

Рантайм освобождает map потомков и разрывает удерживаемые ссылки.

ИНЖЕНЕРНЫЙ ВЫВОД

Состояние завершенного дерева не должно хранить ненужные связи.

cancel() логически мгновенный, физически O(N)

ЗАДАЧА

Одна строка `cancelRoot()` может затронуть десятки тысяч дочерних контекстов.

МЕХАНИЗМ

Под капотом: закрытие Done, рекурсивные cancel, обход и очистка maps.

ИНЖЕНЕРНЫЙ ВЫВОД

Время выполнения cancel нужно измерять отдельно от завершения горутин.

Распространение отмены ≠ окончание работы

ОТМЕНА

`cancel()` закрыл все Done и вернулся.

ЗАВЕРШЕНИЕ

Каждый обработчик еще должен получить CPU, заметить отмену, освободить ресурсы и завершиться.

ИНЖЕНЕРНЫЙ ВЫВОД

Время cancel() и время полного завершения всех работников — разные метрики.

Массовая отмена создает всплеск готовых горутин

ЗАДАЧА

Закрылись Done у 100 000 потомков.

МЕХАНИЗМ

Ожидающие G становятся готовыми; дальше их обрабатывает конечное число M/P.

ИНЖЕНЕРНЫЙ ВЫВОД

Всплеск может проявиться как рост задержек и загрузки CPU уже после cancel.

Зачем вызывать полученный CancelFunc

ЗАДАЧА

Работа завершилась раньше deadline родителя или таймера.

МЕХАНИЗМ

Вызов cancel удаляет запись о потомке у родителя и освобождает связанные ресурсы.

ИНЖЕНЕРНЫЙ ВЫВОД

defer cancel() — ресурсная операция, а не стилистическое украшение.

timerCtx: откуда приходит событие отмены

Timeout — не отдельный протокол, а обычный cancelCtx плюс источник события.

```
cancelCtx + deadline + timer
AfterFunc → DeadlineExceeded
ручная отмена → timer.Stop
```

Из чего состоит timerCtx

СОСТАВ

```
встроенный cancelCtx
deadline — момент времени
timer — источник будущего callback
истечение → cancel(DeadlineExceeded)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Все потомки отменяются обычным уже знакомым механизмом cancelCtx.

Родитель заканчивается раньше requested deadline

GO / РАНТАЙМ

```
ctx, cancel :=
context.WithTimeout(parent,
10*time.Second)
defer cancel()
```

ЧТО ПРОИСХОДИТ

Если дедлайн родителя наступает через 2 секунды, отдельный таймер на 10 секунд не нужен.

ИНЖЕНЕРНЫЙ ВЫВОД

Текущая библиотека выбирает более простой производный контекст, когда срок родителя уже строже.

Дедлайн уже истек

ЗАДАЧА

`time.Until(deadline) <= 0`.

МЕХАНИЗМ

Подключить распространение отмены и немедленно вызвать `cancel(DeadlineExceeded)`.

ИНЖЕНЕРНЫЙ ВЫВОД

Создавать таймер для уже произошедшего события бессмысленно.

Дедлайн в будущем

СОЗДАНИЕ

```
WithDeadline(parent, d)
подключить child к родителю
проверить, что он еще не отменен
time.AfterFunc(duration, callback)
callback → cancel(DeadlineExceeded)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Timer только запускает обычную отмену; не существует отдельного «таймерного дерева».

Ручная отмена останавливает таймер

ЗАДАЧА

Работа закончилась раньше дедлайна.

МЕХАНИЗМ

`timerCtx.cancel` отменяет контекст, удаляет связь и вызывает `timer.Stop()`.

ИНЖЕНЕРНЫЙ ВЫВОД

Не нужно держать таймер до срока после того, как работа уже завершилась.

Стоимость многих WithTimeout

ЗАДАЧА

100 000 запросов в секунду; для каждого создаются независимые дедлайны.

МЕХАНИЗМ

Рантайм регистрирует множество таймеров, а отмены создают дополнительную работу.

ИНЖЕНЕРНЫЙ ВЫВОД

Таймеры и связи контекстов — ограниченный ресурс под высокой нагрузкой.

context.AfterFunc

GO / РАНТАЙМ

```
stop := context.AfterFunc(ctx,
func() {
    cleanup()
})
defer stop()
```

ЧТО ПРОИСХОДИТ

После отмены запускается функция в отдельной горутине. `stop()` пытается предотвратить ее запуск, если он еще не начался.

ИНЖЕНЕРНЫЙ ВЫВОД

Пользовательская функция обратного вызова не исполняется произвольно внутри внутренних локов cancelCtx.

Почему функцию обратного вызова запускают отдельно

ЗАДАЧА

Произвольная функция может блокироваться или выполнять долгую работу.

МЕХАНИЗМ

Рантайм не делает такой вызов синхронной частью защищенной критической секции отмены.

ИНЖЕНЕРНЫЙ ВЫВОД

Нельзя держать внутренний лок, исполняя неизвестный пользовательский код.

WithoutCancel устраняет связь с родителем

GO / РАНТАЙМ

```
detached :=
context.WithoutCancel(parent)
ctx, cancel :=
context.WithCancel(detached)
defer cancel()
```

ЧТО ПРОИСХОДИТ

У `detached.Done() == nil`: стандартный propagateCancel сразу выходит.

ИНЖЕНЕРНЫЙ ВЫВОД

Производный контекст уже не получает отмену через старый parent.

От механизма к цене под нагрузкой

Ищем ограниченный ресурс, симптом и измерение; не оптимизируем API по названию.

```
hchan.lock / копирование значений
широкий select
дерево отмены / таймеры
```

Горячий канал: где заканчивается масштабирование

ЗАДАЧА

Много отправителей и получателей используют один hchan.

МЕХАНИЗМ

Ресурсы: лок канала, очереди ожидания, кеш-линии, пробуждения, пропускная способность памяти.

ИНЖЕНЕРНЫЙ ВЫВОД

Увеличение числа горутин может перестать увеличивать пропускную способность.

Большое значение или указатель в канале

ЗНАЧЕНИЕ

chan [64 << 10]byte
передаем все байты значения.

УКАЗАТЕЛЬ

chan *Large
копируем указатель, но делим время жизни, GC-работу и требования к синхронизации.

ИНЖЕНЕРНЫЙ ВЫВОД

Меньше копирования не означает бесплатную архитектуру.

Большой буфер не делает медленного потребителя быстрым

ЗАДАЧА

Скорость отправки выше скорости получения.

МЕХАНИЗМ

Буфер только позже заполнится; после переполнения отправители снова перейдут к sendq + парковке.

ИНЖЕНЕРНЫЙ ВЫВОД

Увеличение емкости сглаживает всплеск, но не исправляет устойчивый дисбаланс.

Широкий select: где тратится время

ЗАДАЧА

Много case, частая конкуренция за одни и те же каналы.

МЕХАНИЗМ

Подготовка порядков, локи, регистрация sudog и очистка проигравших.

ИНЖЕНЕРНЫЙ ВЫВОД

Отдельно измеряем готовый путь и путь с парковкой.

Обычный Context с большим числом потомков

ЗАДАЧА

Нет отдельной G-наблюдателя на каждую связь.

МЕХАНИЗМ

Но есть `children` map, ее память, операции регистрации/удаления и O(N) обход при cancel.

ИНЖЕНЕРНЫЙ ВЫВОД

Без горутин-наблюдателей — не значит без стоимости.

Пользовательский Context: резкий рост числа горутин

ЗАДАЧА

Реализация не распознается как cancelCtx и не поддерживает AfterFunc.

МЕХАНИЗМ

Каждый новый child может запустить свою горутину-наблюдатель.

ИНЖЕНЕРНЫЙ ВЫВОД

Проверяем профиль горутин и механизм propagateCancel, а не предполагаем утечку сразу.

Диагностика [chan send]

ЗАДАЧА

G заблокирована на отправке.

МЕХАНИЗМ

chansend не нашел получателя и свободного места в буфере; достиг очереди sendq.

ИНЖЕНЕРНЫЙ ВЫВОД

Ищем, почему получатели не освобождают емкость.

Диагностика [chan receive]

ЗАДАЧА

G заблокирована на чтении.

МЕХАНИЗМ

`chanrecv` не нашел ожидающего отправителя, а буфер пуст.

ИНЖЕНЕРНЫЙ ВЫВОД

Проверяем жизненный цикл производителей и закрытие каналов.

Диагностика [select]

ЗАДАЧА

Горутина ждет несколько событий, например job и ctx.Done().

МЕХАНИЗМ

Большое число ожидающих G может быть нормой; подозрителен бесконтрольный рост или отсутствие завершения.

ИНЖЕНЕРНЫЙ ВЫВОД

Всегда сопоставляем состояние ожидания с ожидаемым жизненным циклом.

Инструменты для этих механизмов

ДИАГНОСТИКА

```
профиль горутин → кто и где ждет
профиль блокировок → где и сколько ждали
профиль CPU → chansend / chanrecv / memmove
go tool trace → блокировка → пробуждение → запуск
benchmem → B/op / allocs/op
```

ИНЖЕНЕРНЫЙ ВЫВОД

Каждый инструмент отвечает на свой вопрос; один профиль не объясняет весь путь.

cancel() медленный: что проверяем

ЗАДАЧА

Вызов cancel занимает заметное время.

МЕХАНИЗМ

Ширина дерева, число потомков, обратные вызовы AfterFunc, удерживаемые связи.

ИНЖЕНЕРНЫЙ ВЫВОД

Измеряем сам обход и отдельно задержку завершения рабочих горутин.

G растут после пользовательской обертки Context

ЗАДАЧА

Публичный API WithCancel не изменился, а число G выросло.

МЕХАНИЗМ

Проверяем, не потерялся ли быстрый путь parentCancelCtx и не включился ли резервный путь.

ИНЖЕНЕРНЫЙ ВЫВОД

Одинаковый интерфейс не гарантирует одинаковую цену реализации.

Запустить нагрузить измерить

Эксперименты должны доказывать конкретные ветки алгоритмов, а не повторять определения.

```
канал → обмен → close
select → ширина → пробуждение
context → регистрации → отмена
```

Эксперимент 1: передача напрямую

ЗАДАЧА

Небуферизованный канал, получатель ждет до отправки.

МЕХАНИЗМ

Сравнить T: `uint64`, `[4096]byte`, `[64<<10]byte`, структуру с указателями.

ИНЖЕНЕРНЫЙ ВЫВОД

Измерить ns/op, B/op и профиль CPU: цена копирования зависит от T.

Эксперимент 2: приемник уже ждет или буфер свободен

ЗАДАЧА

Два сценария с одним типом T: ожидающий получатель; либо отправка в пустой буфер.

МЕХАНИЗМ

Сопоставить с ветками `chansend`: прямая передача и `typedmemmove` в `sendx`.

ИНЖЕНЕРНЫЙ ВЫВОД

Синтаксис ch <- v одинаков, но путь рантайма различается.

Эксперимент 3: заполненное кольцо

ВОСПРОИЗВЕСТИ

```
capacity = 2
отправить A, затем B
третий отправитель блокируется с C
получатель читает A
следом из канала идут B, C
```

ИНЖЕНЕРНЫЙ ВЫВОД

Руками пройти: A получатель, C освободившаяся физическая ячейка A.

Эксперимент 4: закрытие и ожидающие G

ЗАДАЧА

Создать N ждущих получателей или M ждущих отправителей.

МЕХАНИЗМ

Вызвать close, зафиксировать разные результаты: zero/false у получателей, panic у отправителей. Изменять N и M.

ИНЖЕНЕРНЫЙ ВЫВОД

Стоимость закрытия растет с числом ожидающих.

Эксперимент 5: число веток select

ЗАДАЧА

Сравнить 2 / 4 / 8 / 16 / 32 / 64 / 128 веток.

МЕХАНИЗМ

Два режима: одна ветка готова и ожидание на всех. Измерить ns/op и профили.

ИНЖЕНЕРНЫЙ ВЫВОД

Отделить первый проход от полной регистрации/очистки.

Эксперимент 6: default и активный опрос

GO / РАНТАЙМ

```
select {
case <-ch:
default:
    // канал пока пуст
}
```

ЧТО ПРОИСХОДИТ

Без default G может припарковаться. С default конструкция немедленно возвращается; цикл без паузы превращается в активный опрос.

ИНЖЕНЕРНЫЙ ВЫВОД

Отсутствие блокировки не означает экономии CPU.

Эксперимент 7: широкий cancel

ЗАДАЧА

Корень с 1 000 / 10 000 / 100 000 потомков.

МЕХАНИЗМ

Измерить длительность `cancelRoot()` и отдельно время до завершения всех работников.

ИНЖЕНЕРНЫЙ ВЫВОД

Синхронная отмена дерева и полный уход горутин имеют разную стоимость.

Эксперимент 8: ленивый Done

GO / РАНТАЙМ

```
a, stopA :=
context.WithCancel(parent)
stopA()

b, stopB :=
context.WithCancel(parent)
_ = b.Done()
stopB()
```

ЧТО ПРОИСХОДИТ

В двух режимах сравнить B/op и allocs/op при контролируемом жизненном цикле.

ИНЖЕНЕРНЫЙ ВЫВОД

Материализация Done — отдельная операция с возможной аллокацией.

Эксперимент 9: стандартный и пользовательский parent

ЗАДАЧА

Одна группа детей создана от стандартного cancelCtx, другая — от Context с собственным Done без AfterFunc.

МЕХАНИЗМ

Сравнить goroutine count, профиль G и аллокации.

ИНЖЕНЕРНЫЙ ВЫВОД

Резервный путь propagateCancel виден руками.

Эксперимент 10: забытый CancelFunc

ЗАДАЧА

Много WithTimeout с длинным дедлайном.
Группа A сразу вызывает cancel; группа B — нет.

МЕХАНИЗМ

Сравнить удерживаемую память и таймерную нагрузку до наступления срока.

ИНЖЕНЕРНЫЙ ВЫВОД

Ранний cancel разрывает связи и освобождает ресурсы раньше.

Эксперимент 11: трассировка массовой отмены

GO / РАНТАЙМ

```
ctx, cancel :=
context.WithCancel(parent)
// много G ждут <-ctx.Done()

cancel()
```

ЧТО ПРОИСХОДИТ

Через go tool trace найти:
ожидающие G закрытие Done
массовую готовность
распределение CPU.

ИНЖЕНЕРНЫЙ ВЫВОД

Возврат cancel и окончательное завершение всех G не совпадают по времени.

Что гарантировано, а что изменится с версией

Сохраняем семантику языка отдельно от конкретных функций и структур рантайма.

```
язык / API → устойчивый контракт
Go 1.27.x → конкретные алгоритмы
под нагрузкой → измеряем стоимость
```

Гарантии и детали реализации

КОНТРАКТ

Язык/API: send/receive, close, select, Done, Err, deadline, CancelFunc, Cause, AfterFunc.

РЕАЛИЗАЦИЯ

Go 1.27.x: hchan, chansend/chanrecv, sendDirect, selectgo, selparkcommit, cancelCtx, parentCancelCtx, timerCtx.

ИНЖЕНЕРНЫЙ ВЫВОД

Прикладной код зависит от контракта; детали текущего рантайма нужны для диагностики.

Полная цепочка канала

МЕНТАЛЬНАЯ МОДЕЛЬ

```
makechan → hchan и буфер
chansend → прямой получатель / свободная ячейка / ожидание
chanrecv → прямое чтение / обмен в полном буфере / ожидание
closechan → разные результаты для recvq и sendq
```

ИНЖЕНЕРНЫЙ ВЫВОД

Одна операция канала может выполнять разную работу в зависимости от состояния.

Полная цепочка select

МЕНТАЛЬНАЯ МОДЕЛЬ

```
pollOrder — справедливая проверка
lockOrder — безопасные локи
проход 1 — уже готовая операция
проход 2 — несколько sudog + selparkcommit
selectDone — победитель
проход 3 — очистка проигравших
```

ИНЖЕНЕРНЫЙ ВЫВОД

Стоимость select зависит от числа веток и необходимости блокироваться.

Полная цепочка context

МЕНТАЛЬНАЯ МОДЕЛЬ

```
WithCancel(parent) → cancelCtx
propagateCancel: нет Done / уже отменен / children / AfterFunc / G
cancel: err + cause + закрыть Done + отменить потомков
timerCtx: событие дедлайна + timer.Stop
```

ИНЖЕНЕРНЫЙ ВЫВОД

Стандартное дерево дешевле модели «горутина на каждого ребенка», но обход тоже не бесплатен.

Главный инженерный вывод

С ВИДУ

Синтаксис мал: `ch <- v`, `select`, `cancel()`.

ПОД НАГРУЗКОЙ

Реальная работа: копирование данных, локи, sudog, сортировка, парковка, обход контекстов, таймеры и пробуждения.

ИНЖЕНЕРНЫЙ ВЫВОД

Ищем горячий механизм, ограниченный ресурс и подтверждаем вывод измерениями.

Мост в 4.3: здесь начинается компилятор

ЗАДАЧА

Мы знаем, что делают `chansend`, `chanrecv` и `selectgo`.

МЕХАНИЗМ

Открытый вопрос: как Go-синтаксис превратился в эти вызовы и какие простые формы компилятор убрал.

ИНЖЕНЕРНЫЙ ВЫВОД

Следующая лекция — IR, SSA, преобразования компилятора, машинный код и измерение производительности.